Amazon API Gateway
Tutorial

Topics

1. Building an API Gateway REST API with Lambda non-proxy integration

Prerequisite

- Complete this tutorial using the AWS Academy Learner Lab dedicated to our module[1].
- Note that if you are completing this tutorial using the AWS Academy Learner Lab, there is a restriction on the **AWS Regions** that can be used, namely all service access is limited to the **us-east-1** and **us-west-2** AWS Regions unless mentioned otherwise.

Build an API Gateway REST API with Lambda integration

Amazon API Gateway is *"a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale"* (source: AWS[2]).
When building an API with Lambda integrations, there are two types of integrations as follows:

- Lambda proxy integration
- Lambda non-proxy integration

In Lambda proxy integration, the input to the integrated Lambda function can be expressed as a combination of path variables, query string parameters, request headers, and body. When using Lambda proxy integration,

[1]This assumes that you have accepted the invitation to join the AWS Academy Learner Lab received via your NCI email account.
[2]https://aws.amazon.com/api-gateway/

an API developer has to choose a certain Lambda function to be integrated. The API developer does not have to configure the integration request and integration response, as they are configured by the Amazon API Gateway.

In Lambda non-proxy integration, an API developer has to ensure that the input to the Lambda function is provided as the integration request payload, i.e. the API developer has to map the input data provided by the client as request parameters in the integration request body, and perform any necessary translations into the form that is acceptable by the backend, in this case, Lambda function.

1 Building an API Gateway REST API with Lambda non-proxy integration

The aim of this tutorial is to learn how to build an API Gateway REST API with Lambda non-proxy integration. More specifically, at the end of this tutorial you should know to perform the following tasks:

- Create a Lambda function for Lambda non-proxy integration
- Create an API with Lambda non-proxy integration
- Test invoking the API method
- Deploy the API
- Test the API in a deployment stage

Please follow the AWS tutorial on building an API Gateway REST API with Lambda non-proxy integration available at https://docs.aws.amazon.com/apigateway/latest/developerguide/getting-started-lambda-non-proxy-integration.html.

In conjunction with the AWS tutorial, please consult the sections below which have similar names with the sections from the AWS tutorial. The sections below provide additional information and guidelines. **Ensure that you follow all the steps from the AWS *Tutorial: Build an API Gateway REST API with Lambda non-proxy integration* provided above (i.e. do not skip sections), rather use this document as an accompanying resource for additional information.**

Create a Lambda function for Lambda non-proxy integration

When creating the Lambda function please ensure the following:

Runtime Instead of following the AWS Tutorial, for *item 4.*, *subitem b.*, ensure that you choose a supported Python runtime.

Execution role If you are using the AWS Lambda servicevia the the AWS Academy Learner Lab, instead of following the AWS Tutorial, for *item 6.*, *subitems c.*, *d.*, and *e.*, ensure that the AWS Lambda service **assumes the *LabRole*** Identity and Access Management (IAM) role. Recall that *LabRole* is a IAM role pre-created for us in the AWS Academy Learner Lab. In the AWS Academy Learner Lab, we need to attach the existing *LabRole* to any AWS Lambda function that needs permission to interact with other AWS services. For example, when creating the Lambda function via the AWS Management Console, ensure that, under the *Change default execution role* section, for the *Execution role* you select the option *Use an existing role* and from the drop-down list you select the *LabRole* role (for reference, please see Figure 1).

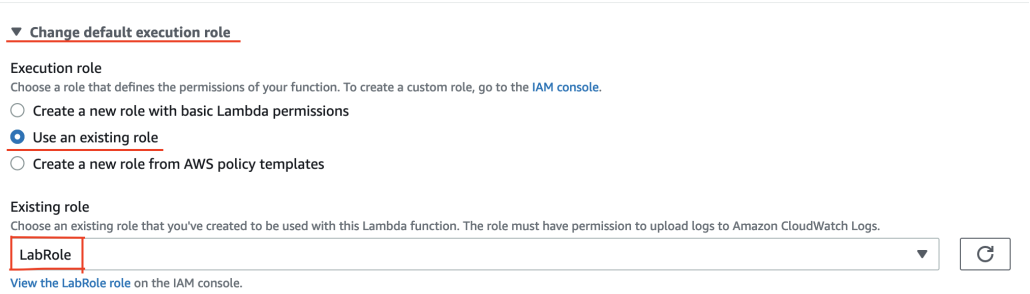


Figure 1: Execution role – *Use an existing role* configuration

Function code Instead of following the AWS Tutorial, for *item 5.*, *subitem a.*, ensure that you use the Python source code provided in this document, available on the next page under the Python Source Code.

Python Source Code

```
import json

def lambda_handler(event, context):

    if event.get('name') == None:
        name = ""
    else:
        name = event['name']

    if event.get('time') == None:
        time = ""
    else:
        time = event['time']

    if event.get('city') == None:
        city = ""
    else:
        city = event['city']

    if event.get('day') == None:
        day = ""
    else:
        day = event['day']


    if name != "":
        greeting = 'Good {}, {} of {}.'.format(time, name, city)
    else:
        greeting = 'Good {}, {}.'.format(time, city)
    if day != "":
        greeting += ' Happy ' + day + '!'

    return {
        'statusCode': 200,
        'greeting': greeting
    }
```

Test the API in a deployment stage

For testing *GET* requests using only URL Path Variables and URL Query String Parameters you can use a browser. Type the API resource URL in a browser. For other methods, use tools such as cURL[3].

For the API created and deployed in this tutorial, test with the following:

- e.g. GET Request using the defined URL Path Variable

```
https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/Dublin
```

where <*api-id*> is the ID of your API, and
<*stage*> is the name of the deployment stage (i.e. the stage where you deployed your API).

- e.g. GET Request using the defined URL Path Variable and URL Query String Parameter

```
https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/Dublin?time=morning
```

where <*api-id*> is the ID of your API, and
<*stage*> is the name of the deployment stage (i.e. the stage where you deployed your API).

To test the API using cURL

Open a terminal window on your computer and run a `cURL` command to test the deployed API.

For the API created and deployed in this tutorial, test with the following

- e.g. to test a POST request

[3] https://curl.se/

```
curl -v -X POST \
'https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/Oslo?time=morning'\
  -H 'content-type: application/json' \
  -H 'day: Wednesday' \
  -H 'x-amz-docs-region: us-east-1' \
  -d '{
    "callerName": "James"
    }'
```

where *<api-id>* is the ID of your API, and
<stage> is the name of the deployment stage (i.e. the stage where you deployed your API).

- e.g. to test a PUT request

```
curl -v -X PUT \
'https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/Oslo?time=morning'\
  -H 'content-type: application/json' \
  -H 'day: Wednesday' \
  -H 'x-amz-docs-region: us-east-1' \
  -d '{
   "callerName": "James"
    }'
```

where *<api-id>* is the ID of your API, and
<stage> is the name of the deployment stage (i.e. the stage where you deployed your API).

Resources

- Amazon API Gateway https://aws.amazon.com/api-gateway/
- Amazon API Gateway – Developer Guide https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html
- Troubleshooting issues with HTTP APIs https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-troubleshooting-lambda.html
- AWS Lambda https://aws.amazon.com/lambda/

- AWS Lambda Developer Guide https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
- Building Lambda functions with Python https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html
- AWS Cloud9 https://docs.aws.amazon.com/cloud9/latest/user-guide/welcome.html
- Python Tutorial for AWS Cloud9 https://docs.aws.amazon.com/cloud9/latest/user-guide/sample-python.html